



Accessible Smartphone Controller for Unity

Colleen Cipriano

December 12, 2025

Contents

1	Introduction	2
1.1	System Overview, Problem Space, and Contributions	2
2	System Setup	4
2.1	Getting Started	4
2.2	Package and File Structure	4
2.3	Setting up the Phone Controller	7
2.4	Running the Main Menu Demo	10
2.5	Running the Exploration Demo	10
3	Integrate ASCU into My Own Game	11
3.1	What to Modify	11
3.2	Drop-in Prefab Setup	12
3.3	Step-by-Step Integration of ASCU into a New Scene	13
3.4	Common Integration Mistakes	14
4	Models and Mathematics	14
4.1	High-Level Architecture	14
4.2	Input Model and Gesture Mapping	15
4.2.1	Touch representation and shared variables	16
4.2.2	Orientation gating	16
4.2.3	Menu-mode gesture classifier	17
4.2.4	Game-mode two-finger model	17
5	Limitations & Future Work	18

1 Introduction

1.1 System Overview, Problem Space, and Contributions

As games become an increasingly central part of modern life, accessibility for blind and low-vision (BLV) players lags behind, leaving many titles hard to use or inaccessible.¹ A common design baseline still assumes (1) continuous access to a high-resolution display, (2) fine motor control over a keyboard/gamepad, and (3) rapid visual parsing of interfaces such as menus, HUDs, prompts, and inventories.² Accessibility research on games repeatedly documents barriers around basic navigation, understanding menu structure, and maintaining control during time-sensitive play.³ In parallel, accessibility guidelines argue that non-visual feedback and alternative input channels can substantially widen access to interactive systems.⁴ This project addresses that gap by developing the **Accessible Smartphone Controller for Unity (ASCU)**, a reusable control-and-feedback layer that allows BLV players to use a smartphone as a controller for a Unity game running on another device. In the current prototype, the Unity side has (i) an accessible main menu demo and (ii) a 2D exploration/platforming demo. The phone side is a web interface that captures familiar touchscreen gestures (swipe, tap, double tap, and continuous holds), transmits them to Unity over a network connection (WebSocket), and receives structured text-to-speech (TTS) prompts and audio cues that describe the game's current state.

ASCU targets two challenges. First, it provides a control scheme that does not rely on a traditional controller or keyboard,⁵ but instead leverages interaction patterns already practiced in everyday smartphone use for BLV users (swiping to navigate, tapping to select, holding to sustain an action).⁶ Second, it makes core interactions understandable without sight by layering an accessibility event system, TTS, and audio cues over existing game logic: gameplay and UI scripts raise semantic events, and a feedback layer turns those events into prioritized narration and cues.⁷ The current implementation focuses on a

¹<https://link.springer.com/article/10.1007/s00500-024-09827-4>

²<https://arxiv.org/html/2502.09866v1>

³<https://www.youtube.com/watch?v=a0zg1VUb61c>

⁴<https://gameaccessibilityguidelines.com/full-list/>

⁵<https://dl.acm.org/doi/10.1145/1978942.1979001>

⁶<https://dl.acm.org/doi/10.1145/2808435.2808448>

⁷<https://gameprogrammingpatterns.com/observer.html>

single-player 2D scenario with obstacles, a checkpoint, and an instruction flow, and it has been successfully adapted to at least one additional 2D Unity game beyond the original demo. ASCU contributes a reusable smartphone controller layer for Unity that enables gesture-driven play without requiring a keyboard or gamepad, a structured accessibility feedback pipeline that converts semantic events into prioritized TTS messages and audio cues without requiring a full rewrite of existing game logic, and a workflow that lowers the barrier to integrating BLV-oriented control and feedback into Unity projects. ASCU provides the following:

- **Phone-to-Unity gesture input over WebSocket:** the phone controller captures touch gestures and sends JSON input events to Unity.
- **Two-mode interaction model:** the controller supports Menu mode (swipe-driven UI navigation and selection) and Game mode (continuous holds for movement and taps for jump/actions), with mode switching triggered by Unity accessibility events.
- **MobileControllerSystem prefab:** bundles the input listener, WebSocket connection, accessibility event router, feedback manager, and a scene bootstrap component for quick setup and reliable mode initialization.
- **Accessibility event pipeline:** Unity scripts raise semantic accessibility events (event ID + default TTS + priority), rather than calling TTS directly, enabling consistent narration across UI and gameplay.
- **Priority-based speech interruption:** incoming feedback is spoken, interrupted, or dropped based on priority so critical messages pre-empt lower-importance narration.
- **Integration pattern for new games:** existing PlayerController scripts can be adapted by subscribing to MobileInputListener events and mapping them into the game's existing movement/jump variables (keyboard fallback optional).

2 System Setup

2.1 Getting Started

Software Requirements

- **Unity Editor:** tested with Unity 2022.3 LTS
- **Web browser on phone:** any browser (Chrome, Safari, Firefox).
- **Local network:** the computer running Unity and the smartphone must be on the same Wi-Fi network.

Package and File Structure

The ASCU is a zipped folder containing two components (Figure 1):

- **Unity side (receiver + feedback):** a Unity project with demo scenes and the system source. This folder also includes an exportable package file (`MobileControllerSystem.unitypackage`) for import.
- **Phone side (controller):** a web app (HTML/CSS/JS) that runs on a smartphone and sends normalized gesture events to Unity.

2.2 Package and File Structure

This subsection lists the important folders in the Feedback Controller folder and the Phone Controller folder (Figure 1). The Unity portion of ASCU lives under `Assets/MobileControllerSystem/`.

- **AccessibilityFeedbackProfiles/:** Scriptable profiles that define how accessibility events should be received by the mobile.
- **Audio/:** Audio clips used by the demo scenes.
- **Documentation/:** Supplemental notes.
- **PhysicsMaterial/:** Physics materials used by the demo gameplay.
- **Prefab/:** Contains `MobileControllerSystem.prefab` that bundles all required runtime components (See 2.5).
- **Scenes/:** Contains the main demo scenes Main Menu (See 2.4) and Exploration Scene Demo (See 2.5).

- **Scripts/**: All system code, organized by purpose:
 - Scripts/MobileNetworking/: WebSocket connection and message handling.
 - Scripts/MobileInput/: Gesture decoding and input state exposed to gameplay/UI.
 - Scripts/EventFeedback/: Accessibility event routing and feedback (TTS/audio) logic.
 - Scripts/MenuUI/: Demo menu navigation and UI wiring.
 - Scripts/DemoGameplay/: Demo-only scripts for the exploration scene.

ASCU uses the open-source [websocket-sharp](https://github.com/sta/websocket-sharp)⁸ library to implement the WebSocket transport layer (client/server) used to exchange JSON messages between the phone controller and Unity.

If you want to *integrate ASCU into a new project*, start in the Prefab/ and connect the phone controller side described in Section 2.3 and skip to Section 3.

⁸<https://github.com/sta/websocket-sharp>

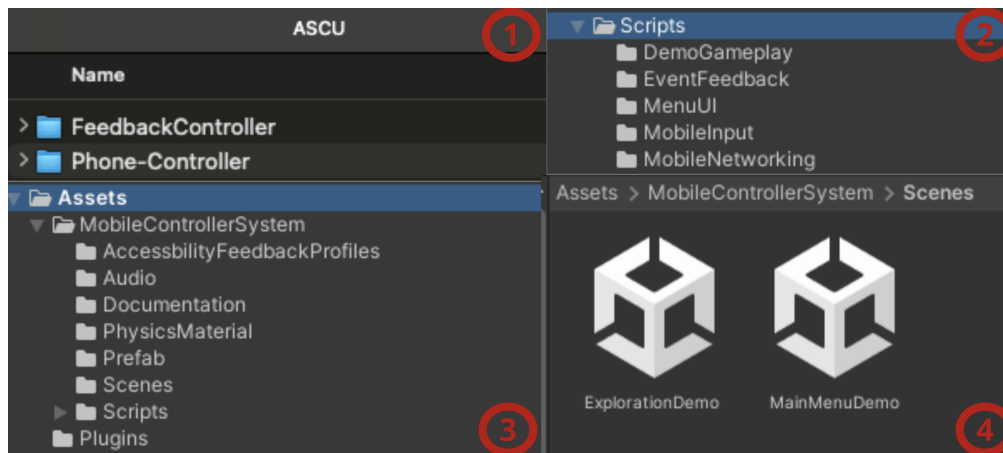


Figure 1: ASCU project layout. (1) Top-level ZIP contents: the Unity folder (FeedbackController) and the phone web controller folder (Phone-Controller). (2) Script organization inside Unity. (3) The imported Unity package root at Assets/MobileControllerSystem/, containing assets, prefabs, scenes, and documentation. Plugins folder containing Web-socket Server (4) Demo scenes included with the package.

Mobile Controller Web App (Phone-Controller folder)

The phone controller web app lives in a separate folder (Figure 1). Its essential files are:

- **controller.html**: the main page loaded on the phone.
- **controller.js**: captures swipes, taps, and holds, and sends them as JSON messages over WebSocket (Fig 4) to the Unity project.
- **controller.css**: basic visual layout, optimized for landscape mode.

Any static web server can host this folder as long as it listens on an address reachable from other devices on your Wi-Fi and your firewall allows the port.

```
public class WebSocketConnect : MonoBehaviour, FeedbackPlug
{
    [Header("Server Settings")]
    [Tooltip("TCP port to listen on for phone controller connections (ws://0.0.0.0:port/).")]
    public int port = 8081;
}
```

Figure 2: Unity-side WebSocket server exposes the listening port (default 8081) in the Inspector; this must match the port the phone client connects to (Figure 4).

2.3 Setting up the Phone Controller

This subsection explains how to get the mobile device talking to Unity. The following steps assume that you have imported the Unity package in your project and you have the files for the Mobile Controller Web App (See Section 2.2). Both devices should be on the same Wi-Fi.

```
▼ TERMINAL
colleencipriano@MacBook-Pro Phone-Controller % python3 -m http.server 8000

::ffff:192.168.1.71 - - [11/Dec/2025 15:59:16] "GET /controller.html HTTP/1.1" 200 -
::ffff:192.168.1.71 - - [11/Dec/2025 15:59:16] "GET /controller.css?v=3 HTTP/1.1" 304 -
::ffff:192.168.1.71 - - [11/Dec/2025 15:59:16] "GET /controller.js?v=3 HTTP/1.1" 304 -
::ffff:192.168.1.71 - - [11/Dec/2025 16:13:12] "GET /controller.html HTTP/1.1" 200 -
::ffff:192.168.1.71 - - [11/Dec/2025 16:13:12] "GET /controller.css?v=1 HTTP/1.1" 200 -
::ffff:192.168.1.71 - - [11/Dec/2025 16:13:12] "GET /controller.js?v=1 HTTP/1.1" 200 -
```

Figure 3: Static server running locally (`python3 -m http.server 8000`). The GET requests confirm the phone successfully loaded `controller.html` and `controller.js` from the correct port.

```
// ----- INPUT (PHONE -> UNITY) ----- //
/**
 * If HARCODED_WS_URL is non-empty, always use that for a tunnel.
 * Otherwise, connect back to the same host that served this page on port 8081.
 */

const HARCODED_WS_URL = "";

function getWebSocketURL() {
  if (HARCODED_WS_URL) {
    return HARCODED_WS_URL;
  }

  const protocol = location.protocol === "https:" ? "wss://" : "ws://";
  return protocol + location.hostname + ":8081";
}

// Resolved WebSocket URL used by connectToUnity()
const WS_URL = getWebSocketURL();
```

Figure 4: How the controller chooses the WebSocket endpoint. If `HARCODED_WS_URL` is empty, the phone connects back to the same host-name that served the page, using port 8081 (and `wss://` when the page is `https://`).

Step 1: Start the Unity Side

1. Load MainMenuDemo.unity from Assets/Scenes/ and select Play.
2. When the scene starts, the prefab called MobileControllerSystem with WebSocketConnect.cs attached will automatically start a WebSocket server and print a message to the Unity Console such as (Fig 2):

```
WebSocket server listening on ws://192.168.0.00:8080
```

Step 2: Start the Mobile Controller Web App

1. On the same machine hosting Unity, start a static web server in that folder (For example, Figure 3):

```
python -m http.server 8000
```

2. On the smartphone, open a browser and navigate to the server URL, **http://<your-computer-ip>/controller.html**

For example:

```
http://192.168.0.00:8000/controller.html
```

Once connected, the web page should look like Figure 5.

3. Enter the desktop/laptop IP address. If you cannot find your IP, select the Connection Help button from the Main Menu scene (See 2.4, #8).
4. When the page loads, it will attempt to open a WebSocket connection to the Unity endpoint (e.g., ws://192.168.0.00:8080). On success, a short message or status indicator on the phone confirms that the controller is connected (Figure 6).

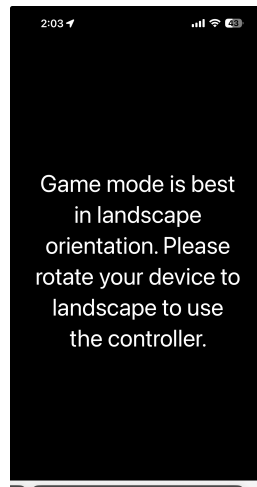


Figure 5: Phone controller successfully loaded after connecting to the host IP/port. The landscape prompt indicates the controller page is running and ready to accept input.

Disconnected. Reconnecting...

Phone Client

Use swipes and taps to navigate the menu and control the game. Audio instructions come from Unity.



Figure 6: Phone controller screen in landscape. The status text indicates the client is attempting to connect/reconnect to the Unity WebSocket server. The `Activate_tts` button is required on many mobile browsers to enable text-to-speech playback. Audio is often blocked until the user performs an explicit interaction.

Step 3: Check Mobile Input

Note (TTS availability varies by browser). Text-to-speech output depends on the mobile browser and OS. If no speech plays after pressing `Activate_tts`, try a different browser (or check system audio settings). Some browsers

restrict speech synthesis until a user gesture occurs, and some may provide limited or unreliable Web Speech support.

Taps and double-taps anywhere act as an activate gesture, depending on the current scene (Figure 6). At this point, basic gestures should already be reaching Unity. You can verify this using the Console output from, which logs events like `Received swipe_up` or `Received double_tap`.

2.4 Running the Main Menu Demo

With the phone connected, the main menu demo guides the user through a simple, accessible flow.

1. Ensure that the Unity Editor is running `MainMenuDemo.unity` in Play mode and the phone controller web page is open and connected.
2. If the controller is loaded but the phone is not rotated, the interface displays the landscape prompt (Figure 5). Rotate the device to enable gameplay gestures.
3. On the phone, you should hear a TTS message if the screen is tapped in portrait mode.
4. Swipe up or down on the phone to move between menu buttons. Each time you move the highlighted button in Unity changes and raises an accessibility event to read out the new selection.
5. Double-tap anywhere on the screen to activate the current selection.
6. Select *Start Game* to load the exploration demo scene.
7. Select *Instructions* if you want to hear the controls of the game.
8. Select *Connection Help* if you want to double check your machine's IP address and if it matches what you entered on your phone's browser.

2.5 Running the Exploration Demo

After selecting *Start* from the main menu, Unity loads the Exploration scene, showing how continuous movement, jumping, and checkpoints work together with accessibility feedback. You can also choose to start from this scene, instead of starting from the Main Menu scene.

1. Confirm that the phone is still connected; if not, refresh the phone controller page or restart the Unity editor.
2. Hold your finger on the left/right side of the screen to move, and tap with another finger to activate jump. Double-tap to make a higher jump.
3. As you progress, reaching a checkpoint triggers a cue and a TTS message such as "Checkpoint reached." This is implemented by `PlayerCollision` raising a "checkpoint_reached" event via the accessibility router.

3 Integrate ASCU into My Own Game

3.1 What to Modify

- **Your movement/UI scripts:** read ASCU input state and apply it to your player or menu navigation.
- **Your gameplay triggers:** raise accessibility events.
- **Your content:** spoken text and audio cues to match.

Subscribe to `MobileInputListener` events and map them into the game's normal movement/jump variables. You typically do *not* need to edit the networking/routing scripts unless you are changing the transport protocol. Figure 7 shows an example patch applied to Unity's 2D Sample project.

```

// --- Phone Controller Integration ---
[Header("Phone Controller (optional)")]
public bool usePhoneInput = false;
public MobileInputListener phoneInput;
public bool allowKeyboardDebug = true;
// Internal phone state
int phoneMoveDir = 0; // -1 = left, 0 = idle, 1 = right
bool phoneJumpPressed = false; // one-shot "jump button down"

protected override void Update() {
    if (controlEnabled) {
        float inputX = 0f;
        bool jumpDown = false;
        bool jumpUp = false;
        // --- PHONE INPUT --- //
        if (usePhoneInput && phoneInput != null) {
            inputX = phoneMoveDir;
            if (phoneJumpPressed) {
                jumpDown = true;
                phoneJumpPressed = false; // consume the event
            }
        }
        // --- KEYBOARD (optional debug / fallback) ---
        if (!usePhoneInput || allowKeyboardDebug) {
            float kbX = Input.GetAxis("Horizontal");
            if (Mathf.Abs(kbX) > Mathf.Abs(inputX))
                inputX = kbX;

            if (Input.GetButtonDown("Jump"))
                jumpDown = true;
            if (Input.GetButtonUp("Jump"))
                jumpUp = true;
        }
        move.x = inputX;

        if (jumpState == JumpState.Grounded && jumpDown) {
            jumpState = JumpState.PrepareToJump;
        } else if (jumpUp) {
            stopJump = true;
            Simulation.Schedule<PlayerStopJump>().player = this;
        } else {
            move.x = 0;
        }
        UpdateJumpState();
        base.Update();
    }
}

protected override void Start()
{
    if (usePhoneInput && phoneInput != null)
    {
        Debug.Log("[PlayerController] Using phone input from " +
            phoneInput.name);

        // Holds for continuous movement
        phoneInput.OnHoldLeftStart += () => phoneMoveDir = -1;
        phoneInput.OnHoldLeftEnd += () => {
            if (phoneMoveDir == -1) phoneMoveDir = 0; };
        phoneInput.OnHoldRightStart += () => phoneMoveDir = 1;
        phoneInput.OnHoldRightEnd += () => {
            if (phoneMoveDir == 1) phoneMoveDir = 0; };

        // Tap / double-tap = jump
        phoneInput.OnTap += OnPhoneJump;
        phoneInput.OnDoubleTap += OnPhoneJump;
    }
}

void OnDestroy()
{
    if (phoneInput != null)
    {
        phoneInput.OnHoldLeftStart -= () => phoneMoveDir = -1;
        phoneInput.OnHoldLeftEnd -= () => {
            if (phoneMoveDir == -1) phoneMoveDir = 0; };
        phoneInput.OnHoldRightStart -= () => phoneMoveDir = 1;
        phoneInput.OnHoldRightEnd -= () => {
            if (phoneMoveDir == 1) phoneMoveDir = 0; };
        phoneInput.OnTap -= OnPhoneJump;
        phoneInput.OnDoubleTap -= OnPhoneJump;
    }
}

void OnPhoneJump()
{
    phoneJumpPressed = true;
}

```

Figure 7: Code snippet added to a PlayerController script of Unity's 2D Sample Project. It listens for hold/tap events from MobileInputListener and translates them into the script's existing movement and jump logic.

3.2 Drop-in Prefab Setup

ASCU is designed to be enabled by placing a prefab in the scene: MobileControllerSystem.prefab. This prefab bundles the required components:

- **MobileInputListener** (receives gestures and exposes input state)
- **WebSocketConnect** (network connection settings, e.g., port)
- **AccessibilityEventsRouter** (central event-raising entry point)
- **FeedbackManager + AudioSource** (TTS/audio feedback and prioritization)
- **GameBootstrap** (helper that sends the event ID to switch the phone controller out of menu mode.)

Some projects do not start from the main menu demo, or they load directly into a gameplay scene. In those cases, the phone controller still needs a clear

signal that the scene is in “game mode” (i.e., it should enable hold-to-move and tap-to-jump instead of menu-only gestures).

3.3 Step-by-Step Integration of ASCU into a New Scene

Step 1: Add the Prefab (bootstraps everything)

Drag `MobileControllerSystem.prefab` into your scene Hierarchy. This single prefab contains the WebSocket connection, gesture input listener, and accessibility feedback manager. Set the **Port** in `WebSocketConnect` if your phone controller uses a different port (default: 8081).

Step 2: Wire your PlayerController to the Phone Input

Your existing `PlayerController` already knows how to move and jump using keyboard input. To support the phone, add a small adapter layer:

- Add three Inspector fields: `Use Phone Input`, `Phone Input`, `Allow Keyboard Debug`,
- Subscribe to `MobileInputListener` events (hold left/right, tap/double tap),
- Map those events into your script’s existing movement/jump variables.

Figure 7 shows an example patch applied to Unity’s 2D Sample project.

Step 3: Test Everything

Press Play and connect the phone controller. A working setup should satisfy all three checks:

1. Swiping/tapping on the phone produces gesture logs in Unity (or visible changes in-game).
2. Hold left/right moves the player continuously; tap triggers jump.
3. When the scene loads, ASCU announces that phone controls are active (via the bootstrap event if enabled).

If Step 2 fails (no movement), the most common cause is that `Phone Input` was not assigned in the `PlayerController` Inspector.

3.4 Common Integration Mistakes

- **No input response:** the prefab is missing from the scene, or the phone is not connected to the correct IP/port.
- **Input connects but movement does nothing:** your movement script is not referencing `MobileInputListener` (or is reading the wrong fields).
- **Speech feels delayed:** long TTS messages are not being interrupted; adjust event priorities or interruption rules in `FeedbackManager`.
- **Works in one scene but not another:** the prefab exists only in the demo scene; add it to every gameplay scene or place it in a bootstrap scene marked `DontDestroyOnLoad`.

4 Models and Mathematics

This section formalizes the phone-side controller logic implemented in `controller.js`. The controller is responsible for (i) capturing touch input, (ii) classifying touches into a small gesture vocabulary, (iii) transmitting gestures to Unity over WebSocket, and (iv) rendering accessibility feedback from Unity via TTS with priority-based interruption.

4.1 High-Level Architecture

ASCU uses a bidirectional message loop between the phone and Unity:

- **Phone → Unity (input):** the phone sends a JSON packet of the form `{type = "input", gesture = g}`, where g is a discrete gesture token (e.g., `swipe_up`, `hold_left_start`, `tap`).
- **Unity → Phone (feedback):** Unity sends accessibility feedback messages containing an event ID, an optional TTS string, and an optional priority value.

In the current design, the phone controller maintains a **state machine**⁹ with two primary modes:

$$M \in \{\text{Menu}, \text{Game}\},$$

⁹<https://gameprogrammingpatterns.com/state.html>

implemented as a boolean (`isInMenu`). Mode is inferred from incoming Unity event IDs: if an event ID begins with the prefix `menu_`, the controller treats the current mode as Menu; otherwise it treats the mode as Game.

The phone controller then captures **touch gestures**¹⁰ and encodes them as JSON commands (e.g., `"swipe_up"`, `"hold_left"`) depending on the mode. These commands are sent over a WebSocket connection to the Unity project.

On the Unity side, `WebSocketConnect.cs` maintains the WebSocket server. Incoming JSON messages are parsed by `MobileInputListener.cs`, which converts them into C# events such as `OnSwipeUp`, `OnSwipeDown`, `OnTap`, and `OnDoubleTap`. Gameplay scripts such as `MainMenuManager.cs` and `PlayerMovement.cs` subscribe to these high-level events instead of dealing with raw network messages.

4.2 Input Model and Gesture Mapping

The phone controller represents user input as a discrete sequence of gesture events. Let

$$G = \{g_1, g_2, \dots, g_N\}$$

denote the sequence of gestures captured over a play session, where each gesture is represented as

$$g_i = (a_i, t_i, m_i),$$

where a_i is the gesture label (e.g., `swipe_up`, `tap`, `hold_left_start`), t_i is the timestamp at which the gesture is detected, and m_i is the metadata (e.g., touch position, duration, device orientation at the time of detection). On the Unity side, these gestures are mapped to abstract input variables for menu navigation and in-game movement:

- **Menu navigation:** swipes change an integer index $k \in \{0, \dots, K - 1\}$ indicating which menu item is selected.
- **Movement:** holds are mapped to a **horizontal axis**¹¹ $x_t \in [-1, 1]$ at discrete time step t .
- **Jump/action:** taps and double-taps are mapped to a jump state $u_t \in \{0, 1\}$, where $u_t = 1$ denotes a jump request, and $u_t = 0$ indicates no jump.

¹⁰<https://w3c.github.io/touch-events/>

¹¹<https://docs.unity3d.com/ScriptReference/Input.GetAxis.html>

In implementation, gameplay scripts do not process raw network messages or touch coordinates; instead, they subscribe to input events exposed by `MobileInputListener`. This separation allows the same gameplay logic to be driven by either the phone controller or conventional inputs by swapping the component that sets movement and jump intent.

4.2.1 Touch representation and shared variables

Each touch gesture begins at position (x_0, y_0) at time t_0 and ends at (x_1, y_1) at time t_1 . Touch position samples are obtained from the browser's [pointer/touch event model](#)¹².

We define displacement and duration as:

$$\Delta x = x_1 - x_0, \quad (1)$$

$$\Delta y = y_1 - y_0, \quad (2)$$

$$\Delta t = t_1 - t_0. \quad (3)$$

Gesture classification in ASCU uses tunable time and distance thresholds, consistent with [common mobile gesture recognizer practice](#)¹³

In the current prototype, the threshold parameters are:

$$D_s = 40 \quad (\text{swipe distance threshold}), \quad (4)$$

$$T_{dt} = 300 \quad (\text{double-tap time window}), \quad (5)$$

$$D_{tap} = 16, \quad T_{tap} = 250 \quad (\text{short tap threshold for gameplay}). \quad (6)$$

These constants are implemented in `controller.js` as `SWIPE_DIST`, `DOUBLE_TAP_MS`, `GAME_TAP_DIST`, and `GAME_TAP_MS`, and can be tuned to match device size and user comfort.

4.2.2 Orientation gating

The controller enforces a hard usability constraint: no gameplay/menu gestures are transmitted unless the phone is in landscape orientation. Landscape is determined by screen geometry:

$$\text{Landscape} \iff W > H,$$

¹²<https://www.w3.org/TR/pointerevents/>

¹³<https://developer.android.com/reference/android/view/ViewConfiguration>

where $W = \text{window.innerWidth}$ and $H = \text{window.innerHeight}$.

If the phone is not in landscape, the system blocks outgoing gestures and instead speaks a rotate instruction when the user performs a small tap (to “unlock” speech and make the hint audible).

4.2.3 Menu-mode gesture classifier

In Menu mode, a single finger anywhere on screen is interpreted as either a swipe or tap/double-tap. The classification rule is based on the displacement magnitude and axis dominance:

- Horizontal swipe if $|\Delta x| > |\Delta y|$ and $|\Delta x| > D_s$
- Vertical swipe if $|\Delta y| > D_s$
- Tap/double-tap if $\Delta t < 250$ ms and displacement is below swipe thresholds

Double-tap is detected using a timestamp memory t_{lastTap} :

$$\text{doubleTap} \iff (t_{\text{now}} - t_{\text{lastTap}}) < T_{dt}.$$

Otherwise the gesture is a single tap and $t_{\text{lastTap}} \leftarrow t_{\text{now}}$.

4.2.4 Game-mode two-finger model

In **Game mode**¹⁴, touch input is split into two roles:

- **Movement finger** (first touch): controls continuous left/right holds.
- **Action finger** (second touch): controls tap/double-tap and optionally swipe-based actions.

This is implemented with two persistent identifiers: `movementTouchId` and `actionTouchId`.

Movement direction selection. Let W be screen width and x be the movement finger touch coordinate at touchstart. Direction is chosen by a simple

¹⁴<https://developer.android.com/develop/ui/views/touch-and-input/gestures/multi>

midpoint test:

$$\text{dir}(x) = \begin{cases} \text{left}, & x < W/2, \\ \text{right}, & x \geq W/2. \end{cases}$$

On movement touchstart, the controller emits `hold_left_start` or `hold_right_start`. On movement touchend, the controller emits `hold_left_end` or `hold_right_end`.

Short movement touches as taps. If the movement finger ends quickly and without significant movement, it is additionally interpreted as a tap/double-tap:

$$(|\Delta x| \leq D_{\text{tap}}) \wedge (|\Delta y| \leq D_{\text{tap}}) \wedge (\Delta t < T_{\text{tap}}).$$

This is how the system supports “tap-to-jump” even when only one finger is used briefly.

5 Limitations & Future Work

This prototype demonstrates the core idea of a smartphone-as-controller plus an accessibility feedback pipeline, but several limitations remain due to platform restrictions and scope constraints.

System Performance

ASCU is event-driven: the phone sends small, JSON messages, which are translated into C# events. The system performs no per-frame polling beyond maintaining the socket and checking queued events, so its steady-state cost is expected to be near-zero; any cost is concentrated at input/feedback events rather than every frame.

Deployment and Packaging Constraints

When the phone controller is hosted on an HTTPS origin (e.g., GitHub Pages or Netlify), modern browsers treat a non-secure WebSocket connection (`ws://`) as *active mixed content* and block it by default. As a result, an HTTPS-hosted controller cannot directly connect to a Unity build that exposes only an insecure local WebSocket server.

For ease of setup, the prototype is designed for local-network use. A production deployment would likely require a more robust distribution model (e.g., native app packaging and secure transport), but these are out of scope for the current prototype.

Platform Limitations: Haptics and Speech Control

The phone controller currently relies primarily on touch input and text-to-speech (TTS) output. In principle, vibration/haptic cues would be a valuable non-visual feedback channel; however, mobile browser support is limited and inconsistent across platforms (especially on iOS), making haptics difficult to implement reliably in a web-based controller. As future work, the controller could be packaged as a native application (or a hybrid app) to access haptic APIs consistently and provide richer tactile feedback.

Similarly, while the controller can trigger TTS, fine-grained control over speech synthesis settings (e.g., voice, rate, pitch, and interruption behavior) is constrained by the browser and operating system. In practice, iOS/Safari imposes stricter limitations than desktop browsers, which restricts how much the prototype can tune speech behavior for different users. Future work includes exposing configurable speech parameters where supported and providing per-device fallbacks. Future work could add:

- configurable haptic patterns (where supported),
- richer spatial or earcon-style audio cues,
- user profiles (e.g., verbosity level, preferred cue types, speech rate),
- localization support for multiple languages and voices.

Gesture and Accessibility Coverage

ASCU currently supports a compact set of gestures (swipes, holds, tap, double tap) chosen for simplicity and learnability. This keeps onboarding easy, but it also limits expressiveness for more complex games (e.g., inventory navigation, or multi-action combat). Future work could introduce an extensible gesture mapping layer (e.g., remappable actions, per-game presets) and evaluate which gesture sets remain usable for BLV players under time pressure.

Evaluation Scope and Generalizability

The current implementation focuses on a 2D single-player exploration/platforming scenario with an accessible main menu and a small set of gameplay events. While the system has been adapted to an additional 2D game context, it has not yet been systematically evaluated across diverse genres.